care of calling the move constructor. The `process_data()` function can then transfer ownership directly into its own `std::unique_lock` instance ❷, and the call to `do_something()` can rely on the data being correctly prepared without another thread altering the data in the meantime.

Typically this sort of pattern would be used where the mutex to be locked is dependent on the current state of the program or on an argument passed in to the function that returns the `std::unique_lock` object. One such usage is where the lock isn't returned directly but is a data member of a gateway class used to ensure correctly locked access to some protected data. In this case, all access to the data is through this gateway class: when you wish to access the data, you obtain an instance of the gateway class (by calling a function such as `get_lock()` in the preceding example), which acquires the lock. You can then access the data through member functions of the gateway object. When you're finished, you destroy the gateway object, which releases the lock and allows other threads to access the protected data. Such a gateway object may well be movable (so that it can be returned from a function), in which case the lock object data member also needs to be movable.

The flexibility of `std::unique_lock` also allows instances to relinquish their locks before they're destroyed. You can do this with the `unlock()` member function, just like for a mutex: `std::unique_lock` supports the same basic set of member functions for locking and unlocking as a mutex does, in order that it can be used with generic functions such as `std::lock`. The ability to release a lock before the `std::unique_lock` instance is destroyed means that you can optionally release it in a specific code branch if it's apparent that the lock is no longer required. This can be important for the performance of the application; holding a lock for longer than required can cause a drop in performance, because other threads waiting for the lock are prevented from proceeding for longer than necessary.

3.2.8. Locking at an appropriate granularity